

FPL-AI

Anbefalinger og videre arkitektur — status per august 2026

0. Er grunnlaget riktig?

Ja, i hovedsak. Todelingen mellom Python (beregning, data, søk) og en AI-modell (tolkning, kontekst, forklaring) er en sunn arkitektur, og dere har allerede identifisert det viktigste problemet selv: brute-force-optimizeren (V1–V5) skalerer ikke. Det som mangler er ikke flere features først — det er en optimizer-kjerne som faktisk er rask og korrekt, pluss en tydelig kontrakt for hvordan Python og AI-laget snakker sammen. Resten av dokumentet er strukturert som en anbefalt rekkefølge, ikke bare en liste.

1. Løs optimizer-kjernen først (høyest prioritet)

Alt V1–V5 har til felles er at dere prøver å generere og filtrere kandidater før dere kombinerer posisjonsgrupper. Det er dette som gir kombinatorisk eksplosjon. Løsningen er å bytte metode, ikke å stramme filtrene enda mer.

Anbefaling: bruk lineær heltallsoptimering (MILP / ILP)

Squad-valg i FPL er et klassisk "knapsack med constraints"-problem. Med biblioteker som PuLP, OR-Tools (CP-SAT) eller mip kan et komplett 15-mannslag med budsjett, posisjonskrav, maks 3 spillere per Premier League-lag og formasjonskrav løses eksakt på under ett sekund — for hele spillerpoolen på ~600–700 spillere, uten å redusere kandidatlisten manuelt først.

Kjernen i en slik modell:

- **Beslutningsvariabler:** én binær variabel per spiller (velges/velges ikke), evt. egen variabel for startellever vs. benk og for kaptein.
- **Objective:** maksimer sum av forventede poeng (vektet over flere GW, se punkt 3).
- **Constraints:** budsjett ($\leq 100m$), 2 keeper / 5 forsvar / 5 midtbane / 3 angrep, maks 3 spillere fra samme PL-lag, gyldig startformasjon (f.eks. 3-4-3 til 5-2-3-varianter).
- **Transfer-modus:** når det ikke er wildcard, legg til constraint på antall endringer fra nåværende lag + straff for ekstra transfers utover frikvote.

Dette gjør at hele "generer kandidater → kombiner grupper"-steget i V1–V5 kan fjernes helt. Pandas/DataFrame skal kun brukes til datavask og feature-bygging — aldri inne i selve søket.

2. Datagrunnlag som bør legges til

FPL API alene gir ikke nok til å modellere "hvordan spillere faktisk scorer" slik notatene deres beskriver (DefCon, xG/xA, set-piece-trussel). Dette må hentes fra andre kilder:

- **xG / xA / skudd i boks:** Understat eller FBref (scraping) — eller en betalt kilde som Opta/FPL Review-API hvis budsjett tillater.
- **Defensive contribution (DefCon):** må beregnes fra rå matchdata (taklinger, klareringer, recoveries) siden FPL API ikke gir dette direkte per spiller ennå.
- **Set-piece-info:** hvem som tar corner/frispark/straffer — finnes ikke strukturert i FPL API, må hentes manuelt eller fra en fotball-datakilde.

- **Skadedata / nyheter:** FPL sin "news"-felt er ofte forsinket. Vurder en enkel scraping av Premier Injuries eller lignende som ekstra signal til xMins-modellen.
- **Fixture-vanskelighet:** ikke bruk FPL sin egen FDR (for grov). Bygg egen basert på xG scored/conceded per lag, splittet hjemme/borte.

3. Prediksjonsmodell — forbedringer

xMins som egen modell

Behandle xMins som en liten klassifiserings-/regresjonsmodell (f.eks. gradient boosting) trent på minutter siste N kamper, rotasjonsmønster, skadestatus og lagets kampprogram — ikke regelbasert logikk slik V1 var. Dette løser "for mange VERY_HIGH"-problemet dere beskriver på en mer robust måte enn manuell justering.

Multi-Gameweek med fallende vekt

Planen deres (GW+1 høyest vekt, fallende mot GW+5) er riktig. I MILP-objektivet blir dette ganske enkelt en vektet sum over horisonten, f.eks. vekt 1.0 / 0.7 / 0.5 / 0.35 / 0.25.

Usikkerhet, ikke bare forventningsverdi

En spiller med 6.0 forventede poeng ± 1 er ikke det samme som ± 4 . Legg til et enkelt usikkerhetsmål (f.eks. varians i historiske poeng, eller prediction interval fra modellen) og la optimizere kunne straffe høy varians for "trygge" ukesvalg, eller premiere det for differensial-strategi. Dette mangler helt i notatene og er en reell forbedring, ikke bare kompleksitet for kompleksitetens skyld.

4. Funksjonsområder som bør legges til

- **Price change-prediksjon:** mange etablerte FPL-verktøy (f.eks. LiveFPL) predikerer prisendringer neste natt basert på transfer-momentum. Nyttig for timing av transfers før en spiller stiger/faller i pris.
- **Ownership / Effective Ownership (EO):** avgjørende for kaptein- og differensial-beslutninger — mangler helt fra notatene deres per nå. FPL API gir global ownership-tall dere allerede har tilgang til.
- **Free Hit- og Bench Boost-spesifikk logikk:** notatene nevner kun wildcard eksplisitt. Free Hit trenger egen ettGW-optimalisering uten hensyn til fremtidige GW, Bench Boost krever egen scoring av benkespillere.
- **Egen "differensial-modus":** et valg i optimizere mellom "trygt/konsensus-lag" og "høy-EO-avvik", nyttig i ligaer der man ligger bak og trenger risiko.

5. Backtesting — gjør dette før dere stoler på systemet

Dere har rett i at dette er kritisk. Konkret opplegg:

- 1 Kjør prediksjonsmodellen på historiske gameweeks (f.eks. hele forrige sesong) og sammenlign predicted vs. actual points per spiller, brutt ned på posisjon.
- 2 Kjør optimizere på historiske data GW-for-GW som om den var "live", og sammenlign faktisk oppnådd poengsum mot topp-mannlag/gjennomsnitt for samme periode.
- 3 Test xMins-modellen separat: predicted vs. actual minutter, spesielt rundt rotasjon og retur fra skade.
- 4 Test kapteinmodellen isolert — hvor ofte traff den blant topp-3 kaptein-valgene den uken.
- 5 Loggfør alt i en enkel CSV/dashboard slik at forbedringer kan måles over tid, ikke bare "føles bedre".

6. Hvordan programmet bør samhandle med AI

Ikke bygg en exe. Poenget med "AI har ikke tilgang til alt av statistikk" løses ikke av pakkeformat (exe vs. terminal) — det løses av hvordan dere strukturerer kallet til AI-API-et. Anbefalt mønster:

- Python-pipelinen kjører ferdig (data → features → predictions → optimizer-resultat).
- Resultatet pakkes i et strukturert JSON-objekt (lag, budsjett, forslag til transfers, kaptein-kandidater, usikkerhet, chip-status).
- Dette sendes som kontekst i ett API-kall til språkmodellen, som gir tolkning/forklaring/anbefaling tilbake i naturlig språk.
- AI-en "vet" da akkurat det dere velger å sende — ikke mer, ikke mindre. Dette er den reelle løsningen på tilgangsproblemet, uavhengig av om appen kjøres i terminal, Streamlit eller som exe.

Om grensesnitt

Bli i terminal så lenge dere er eneste bruker. Vurder Streamlit eller Gradio hvis dere vil ha knapper/tabeller/chat-vindu uten packaging-arbeid. Exe er kun relevant den dagen verktøyet skal deles med noen uten Python installert — ikke en prioritet nå.

7. Prosjekthygiene

- Rydd opp V1–V5 til én tydelig "current" versjon i GitHub-struktur — behold gamle versjoner kun som referanse i en /legacy-mappe eller git-historikk, ikke i hovedmappen.
- Skriv dependencies/setup-dokumentasjon (requirements.txt + kort README om pipeline-rekkefølge).
- Legg til enkle enhetstester for feature-bygging og optimizer-constraints (f.eks. "løsningen bryter aldri budsjett", "aldri mer enn 3 fra samme lag").
- Bygg én samlet weekly runner som kjører hele pipelinen i riktig rekkefølge automatisk, i stedet for manuelle scriptkjøringer.

8. Anbefalt prioritert rekkefølge

#	Tiltak	Hvorfor
1	Bytt optimizer til MILP (PuLP/OR-Tools)	Løser flaskehalsen som har blokkert V1–V5
2	Definer JSON-kontrakt mellom Python-pipeline og AI-lag	Gjør "samhandling" konkret og gjenbrukbar
3	Legg til xG/xA/DefCon/set-piece-data fra ekstern kilde	Nødvendig for posisjonsspesifikk scoring
4	Bygg xMins som ordentlig modell, ikke regelbasert	Løser rotasjons-/VERY_HIGH-problemet
5	Legg til usikkerhet + ownership/EO i optimizer-input	Gir bedre kaptein- og differensial-beslutninger
6	Backtesting-rammeverk	Bekrefter at systemet faktisk er godt, ikke bare "ser smart ut"

7	Weekly runner + opprydding av kodebase	Gjør systemet driftbart uke etter uke
---	--	---------------------------------------

Kort sagt: arkitekturen og forståelsen deres er allerede riktig — det som gjenstår er å bytte søkemetode i optimeren, koble på bedre underliggende data, og gjøre AI-samhandlingen til en eksplisitt datakontrakt fremfor et pakkeformat-spørsmål.